Assignment 2 Reflection

Name: Bhavesh Upadhyaya **Option:** Option A - Google Calendar Assistant **Date:** October 9, 2026

Tool Design Decisions

Tools Implemented

1. `list_upcoming_events` : Retrieves scheduled calendar events within a specified date window, ordered chronologically so the agent can quickly summarize upcoming commitments.
2. `check_conflicts` : Evaluates proposed meeting windows against existing calendar events using overlap logic to catch double-bookings before any schedule changes are made.
3. `find_available_slots` : Scans the user's calendar across a date range and identifies open, non-overlapping 60-minute time blocks during standard business hours (9:00 AM – 5:00 PM).
4. `create_event` : Formats and inserts a new calendar event with summary, start/end timestamps, optional description, meeting location, and attendee emails.
5. `reschedule_event` : Modifies start and end times for an existing event by its event ID while preserving all existing metadata (summary, description, attendee list, meeting links).

Why These Tools?

I chose Option A because calendar management is an immediate, high-friction problem in my day-to-day workflow. I balance multiple personal and non-profit group email accounts alongside several distinct consulting clients and work commitments. My ultimate goal is to build a safe, privacy-preserving calendar blocker: when a meeting gets scheduled with one client or organization, the agent should automatically create a private "busy" placeholder on my other calendars without exposing client-confidential meeting titles or details.

To make that feasible, an assistant needs more than just a blind `create_event` tool. It needs discovery (`list_upcoming_events`), proactive conflict detection (`check_conflicts`), and free-slot discovery (`find_available_slots`) so that neither I nor the agent double-books time.

Description Strategy

Writing tool descriptions for an LLM is essentially writing API documentation for a neural network. If descriptions are vague, the agent guesses or picks the wrong tool.

- I used explicit action verbs (`list` , `check` , `find` , `create` , `reschedule`) right at the start of function names and descriptions.
- I included trigger phrases explaining *when* to use each tool (e.g., "Use this tool before creating or rescheduling a meeting to verify whether the user already has another appointment").
- I documented explicit ISO-8601 parameter examples (`YYYY-MM-DDTHH:MM:SSZ`) so the LLM provides standardized arguments rather than ambiguous date strings like "next Tuesday afternoon".

Challenges Encountered

Challenge 1: The `adk web` Packaging vs. Script-Mode Conflict

- **Problem:** Running `adk web` from inside the folder triggered `ModuleNotFoundError: No module named 'config'` and failed to locate the agent. Initially, I thought this should be a simple code fix—it's obvious the config line is right there in the file, so it must not be referenced correctly. The issue stemmed from the difference between running `python main.py` (which treats the local folder as `sys.path[0]`) versus running `adk web` (which dynamically treats `workspace_assistant` as a package from the parent directory). Furthermore, ADK's loader specifically looked for a module-level variable named `root_agent` rather than just the `create_agent()` factory function.
- **Solution:** We resolved this cleanly without modifying any of the provided starter files. We exposed `root_agent = create_agent()` in `agent.py` , added `workspace_assistant/__init__.py` , and applied dynamic `sys.path` normalization with fallback imports (`try: from config... except ImportError: from workspace_assistant.config...`). Once fixed, using `adk web` made understanding the agent's workflow significantly clearer: the Web UI provides a visual representation of agent calls, execution traces, and individual spans that are far easier to inspect and debug than raw terminal logs.

Challenge 2: Datetime Formatting and API Boundaries

- **Problem:** Google Calendar API v3 is unforgiving: passing naive timestamps without timezone offsets or date-only strings without midnight boundaries causes immediate `HTTP`

`400 Bad Request` errors.

- **Solution:** Implemented `_parse_iso_datetime` inside `calendar_tools.py`. It catches common user inputs (like `YYYY-MM-DD` or space-separated dates), normalizes them to ISO-8601, and defaults naive timestamps to UTC before sending payloads across the network.

Challenge 3: GitHub Classic Token Scope Confusion

- **Problem:** The assignment setup documentation instructed creating a GitHub Classic Token and selecting `write:issues`. However, `write:issues` does not exist in GitHub's Classic token interface—it only exists in newer fine-grained tokens.
- **Solution:** In GitHub Classic tokens, issue creation and updates on public repositories are bundled directly inside the `public_repo` scope. Selecting `public_repo` gave the MCP server the exact permissions needed to list repositories, query issues, and create new issues without granting unnecessary account-level access.

Error Handling Approach

My philosophy on error handling for autonomous agents is centered around communication and safety:

1. **Mandatory Confirmation Before Action:** If we don't confirm meeting details prior to creating or rescheduling, we run a severe risk of corrupting or missing up-to-date metadata like attendee email lists, meeting links, or agenda notes. The system instructions explicitly require the agent to confirm the summary, date, start time, end time, and attendees with the user before finalizing any calendar modification.
2. **Graceful Error Trapping:** Every tool wraps external API interactions in a `try...except` block and returns a dictionary: `{"status": "error", "message": "..."}`. Letting Python throw an unhandled traceback crashes the entire runner. By returning structured error payloads, the LLM reads the failure message and translates it into an actionable, polite explanation for the user.

Bonus: Tool Search Pattern & Empirical Context Bloat Measurement

Reducing context bloat is paramount to keeping an agent's total context window manageable. We don't want to use up precious context window capacity for extensive tool schemas that do

not apply to the user's immediate question. Tokens—even when cached—take up valuable window space, add processing latency as the model reads and ignores them, and incur real financial costs on every API call.

Rather than relying on unmeasured hypothetical estimates, I empirically measured the token consumption across both agent configurations using the exact same query:
"What meetings do I have scheduled for today and check if there are open issues on my repository?"

Empirical Measurement Setup & Results

- **Model Used:** `gemini-3.8-flash`
- **ADK Framework Version:** `google-adk==2.11.0`
- **Runtime:** Python 3.13 on macOS
- **Measurement Metric:** `event.usage_metadata.prompt_token_count` from live `InMemoryRunner` turn events

Agent Configuration	Tools Loaded Upfront	Measured Prompt Tokens	Measured Total Tokens	Token Reduction
Standard Agent (Part 1 + 2)	All 15+ GitHub MCP tools + 5 Calendar tools	**19,105 tokens**	19,884 tokens	Baseline
Tool Search Agent (Bonus)	2 Primary GitHub tools + `search_github_tools` + 5 Calendar tools	**8,218 tokens**	8,885 tokens	**-10,887 tokens (56.99% savings)**

Implementation Architecture

Because ADK has no native `defer_loading` parameter (which belongs to Anthropic's Messages API), I implemented the dynamic tool search pattern using ADK's native primitives:

1. **Minimal Upfront Toolset (5 pts):** Configured `DeferredMcpToolset` with a custom `ToolPredicate` (`dynamic_github_tool_filter`) that exposes at most two essential tools upfront (`search_repositories` and `list_issues`).
2. **Stateful Tool Search (5 pts):** Implemented `search_github_tools(query, tool_context)` which queries the catalog, returns each matched tool's name, description,

and input schema, and automatically saves newly discovered tool names into `tool_context.state["unlocked_tools"]`.

3. **Immediate Tool Usability & ADK 2.x Invalidation (8 pts):** In ADK 2.x, the framework caches tool definitions per invocation (`_use_invocation_cache = True`). By explicitly setting `toolset._use_invocation_cache = False`, tools unlocked in session state by `search_github_tools` become immediately visible and callable by the agent within the very same reasoning turn.
4. **Agent Factory (2 pts):** Provided `create_agent_with_tool_search()` in `agent.py` to package this dynamic architecture cleanly.
5. **Empirical Verification (5 pts):** The verified ~57% token reduction (saving nearly 11,000 prompt tokens per turn) demonstrates the tangible efficiency gains of lazy tool discovery in production agents.

Ideas for Improvement

If I had more time to extend this project, I would focus on solving my real multi-calendar challenge:

1. **Multi-Account & Microsoft Exchange Integration:** Build out an agent set that can connect to Microsoft Exchange / Outlook in addition to Google Calendar, allowing the agent to manage time and meeting blocking across 5 different consulting, non-profit, and personal accounts simultaneously.
2. **Automated Cross-Calendar Busy Placeholders:** Implement an automated synchronization worker where an accepted meeting on one client calendar automatically creates an anonymized "Busy / Focus Time" placeholder on all other accounts to prevent double-booking while preserving client confidentiality.
3. **Smart Conflict Resolution Suggestions:** When a conflict is detected, have the agent proactively query all relevant calendars and propose the top three alternative slots that work for all attendees.

Key Learnings

Reflecting on Week 1 versus Week 2, multi-agent orchestration via Crews (like CrewAI) felt somewhat rudimentary. In that model, you practically need "armies" of specific crews, each hardcoded with their own unique set of tools. You could try connecting MCP tools to individual

crews, but then tool management happens at an fragmented crew level rather than a clean, centralized system level, making orchestration much harder to maintain.

In contrast, the Google ADK and MCP model feels far more practical and scalable for production engineering. Giving an agent access to a standardized suite of self-updating tools via protocols like MCP means dramatically less custom glue code and vastly superior interoperability across platforms (Google Workspace, GitHub, and beyond). Once you understand how to manage context bloat with patterns like deferred loading, building autonomous agents around standardized protocols is a much more robust architectural pattern.